

**LAPORAN PRAKTIKUM SISTEM MANAJEMENT PROSES
OPERASI SYSTEM**



**Dosen Pengampu :
Triawan Adi Cahyanto M.kom**

**Disusun Oleh :
Hafabi Arrohim (2410651013)**

**PROGRAM STUDY TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
TAHUN 2025**

1. Tujuan

Tugas-tugas yang ingin dicapai dalam praktikum ini adalah pemahaman konsep dasar dari manajemen proses dalam sistem operasi, yaitu:

- Membuat dan mengelola beberapa child process menggunakan fungsi `fork()`.
- Mengimplementasikan tugas yang berbeda pada setiap proses yang berbeda : SORTING, SEARCHING, CALCULATION.
- Mengkoordinasikan hasil antar semua proses melalui parent process.
- Menampilkan informasi proses seperti PID, status, waktu eksekusi.

2. Dasar Teori

Dalam sistem operasi, proses adalah program yang sedang dieksekusi.

Fungsi utama yang digunakan dalam manajemen proses pada bahasa C antara lain:

- `fork()` → digunakan untuk membuat proses baru (child process).
- `getpid()` → mendapatkan Process ID dari proses yang sedang berjalan.
- `wait()` atau `waitpid()` → membuat parent menunggu child selesai.
- `exit()` → mengakhiri proses anak.
- `clock()` → menghitung waktu eksekusi program.
- Konsep ini menggambarkan cara sistem operasi melakukan multiprocessing dan inter-process coordination (IPC) sederhana.

3. Pecobaan

a. Source Kode Lengkap

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
#include <time.h>

void sorting_task() {
    int arr[] = {5, 2, 9, 1, 3};
    int n = 5, i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    printf("Child 1 (PID %d): Hasil sorting: ", getpid());
    for (i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

void searching_task() {
    int arr[] = {3, 6, 9, 12, 15};
    int n = 5, target = 9, i, found = 0;
    for (i = 0; i < n; i++) {
        if (arr[i] == target) {
            found = 1;
            break;
        }
    }
}

```

```

    }
}
printf("Child 2 (PID %d): Searching angka %d -> %s\n", getpid(), target, found ? "Ditemukan" : "Tidak ditemukan");
}

void calculation_task() {
    int i, sum = 0;
    for (i = 1; i <= 5; i++) sum += i;
    printf("Child 3 (PID %d): Hasil penjumlahan 1-5 = %d\n", getpid(), sum);
}

int main() {
    pid_t pid1, pid2, pid3;
    clock_t start, end;
    double cpu_time_used;

    printf("=== SISTEM MANAJEMEN PROSES ===\n");

    start = clock();

    pid1 = fork();
    if (pid1 == 0) {
        sorting_task();
        exit(0);
    } else {
        pid2 = fork();
        if (pid2 == 0) {
            searching_task();
            exit(0);
        } else {
            pid3 = fork();
            if (pid3 == 0) {
                calculation_task();
                exit(0);
            } else {
                // Main process continues here
            }
        }
    }

    end = clock();
    cpu_time_used = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Total CPU Time Used: %f seconds\n", cpu_time_used);
}

```

```

pid1 = fork();
if (pid1 == 0) {
    sorting_task();
    exit(0);
} else {
    pid2 = fork();
    if (pid2 == 0) {
        searching_task();
        exit(0);
    } else {
        pid3 = fork();
        if (pid3 == 0) {
            calculation_task();
            exit(0);
        } else {
            int status;
            waitpid(pid1, &status, 0);
            waitpid(pid2, &status, 0);
            waitpid(pid3, &status, 0);
            end = clock();

            cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
            printf("\nParent (PID %d): Semua proses selesai!\n", getpid());
            printf("Waktu eksekusi total: %.4f detik\n", cpu_time_used);
        }
    }
}

return 0;
}

```

Jalankan Program :

```

faby@fabyar-biee:~/Tugas_Proses$ nano sistem_proses.c
faby@fabyar-biee:~/Tugas_Proses$ gcc sistem_proses.c -o sistem_proses
faby@fabyar-biee:~/Tugas_Proses$ ./sistem_proses
=== SISTEM MANAJEMEN PROSES ===

```

Output program :

```

faby@fabyar-biee:~/Tugas_Proses$ ./sistem_proses
=== SISTEM MANAJEMEN PROSES ===
Child 1 (PID 565): Hasil sorting: 1 2 3 5 9
Child 2 (PID 566): Searching angka 9 -> Ditemukan
Child 3 (PID 567): Hasil penjumlahan 1-5 = 15

Parent (PID 564): Semua proses selesai!
Waktu eksekusi total: 0.0013 detik
faby@fabyar-biee:~/Tugas_Proses$ |

```

b. Analisis Hasil

Berdasarkan hasil output, program berhasil membuat tiga proses anak yang menjalankan tugas berbeda:

- Proses Tugas Output Status
- Child 1 Sorting array Menampilkan hasil urutan angka Selesai
- Child 2 Searching angka Menampilkan hasil pencarian Selesai
- Child 3 Perhitungan (sum) Menampilkan hasil penjumlahan Selesai
- Parent Koordinasi proses anak Menampilkan waktu eksekusi total
Selesai
- Setiap proses memiliki PID unik, menunjukkan bahwa proses dijalankan terpisah.

- Parent menunggu seluruh child selesai melalui fungsi `waitpid()`, sesuai konsep synchronization antar proses.

c. Jawaban Tugas

Program berhasil membuat multiple child processes.

Setiap child menjalankan tugas berbeda (sorting, searching, calculation).

Parent mengumpulkan hasil dan menampilkan PID, waktu, dan status.

```
faby@fabyyar-biee:~/Tugas_Proses$ ./sistem_proses
=== SISTEM MANAJEMEN PROSES ===
Child 1 (PID 565): Hasil sorting: 1 2 3 5 9
Child 2 (PID 566): Searching angka 9 -> Ditemukan
Child 3 (PID 567): Hasil penjumlahan 1-5 = 15

Parent (PID 564): Semua proses selesai!
Waktu eksekusi total: 0.0013 detik
faby@fabyyar-biee:~/Tugas_Proses$ |
```

d. Analisis Performa

Waktu eksekusi total menunjukkan efisiensi koordinasi antar proses.

Karena setiap proses sederhana, waktu eksekusinya kecil (< 0.01 detik).

Namun, dalam sistem nyata, pembuatan proses tambahan menambah overhead CPU dan memori.

4. Kesimpulan

- Program berhasil mensimulasikan sistem manajemen proses dengan tiga child process yang menjalankan tugas berbeda.
- Fungsi `fork()` dan `waitpid()` bekerja untuk membuat dan menyinkronkan proses.
- Hasil menunjukkan setiap proses memiliki PID unik dan status “selesai”.
- Praktikum ini membantu memahami dasar proses paralel dan koordinasi antar proses (IPC) di sistem operasi.

